SQL to PySpark Phase 3- ETL

Reading the CUSTOMERS AND SALES DATA

1. Load Customer and Order Data

In [ ]:
```
# Load customer and order data from volumes
customers = spark.read.option("header",
"true").csv("/Volumes/workspace/default/customers")
orders = spark.read.option("header",
"true").csv("/Volumes/workspace/default/customers/orders.csv")
print(f"Customers loaded: {customers.count()} rows")
print(f"Orders loaded: {orders.count()} rows")
```

```
Customers loaded: 160 rows
Orders loaded: 60 rows
```

2. Explore Data Schema and Structure

In [ ]:
```
# Display schema for both datasets
print("=== CUSTOMERS SCHEMA ===")
customers.printSchema()
print("\n=== ORDERS SCHEMA ===")
orders.printSchema()
print("\n=== SAMPLE DATA ===")
display(customers.limit(5))
display(orders.limit(5))
```

```
=== CUSTOMERS SCHEMA ===
root
 |-- customer_id: string (nullable = true)
 |-- first_name: string (nullable = true)
 |-- last_name: string (nullable = true)
 |-- email: string (nullable = true)
 |-- phone_number: string (nullable = true)
 |-- address: string (nullable = true)
 |-- city: string (nullable = true)
 |-- state: string (nullable = true)
 |-- zip_code: string (nullable = true)


=== ORDERS SCHEMA ===
root
 |-- order_id: string (nullable = true)
 |-- customer_id: string (nullable = true)
 |-- order_date: string (nullable = true)
 |-- status: string (nullable = true)
 |-- total_amount: string (nullable = true)


=== SAMPLE DATA ===
```

customer_id	first_name	last_name	email	ph
1	John	Smith	john.smith@domain.com	55
2	Emma	Jones	emma.jones@webmail.com	55
3	Olivia	Brown	olivia.brown@outlook.com	55
4	Liam	Johnson	liam.johnson@gmail.com	55

order_id	customer_id	order_date	status	total_amount
1	1	2024-01-15	Shipped	39.98
2	1	2024-01-16	Pending	29.99

order_id	customer_id	order_date	status	total_amount
3	2	2024-01-17	Delivered	25.0
4	2	2024-01-18	Shipped	89.97
5	3	2024-01-19	Pending	49.98

3. Identify Missing Values

In [ ]:
```
from pyspark.sql.functions import col, sum as spark_sum

# Count missing values per column for customers
print("=== CUSTOMERS - Missing Values ===")
customers_missing =
customers.select([spark_sum(col(c).isNull().cast("int")).alias(c) for c
in customers.columns])
display(customers_missing)

# Count missing values per column for orders
print("\n=== ORDERS - Missing Values ===")
orders_missing =
orders.select([spark_sum(col(c).isNull().cast("int")).alias(c) for c in
orders.columns])
display(orders_missing)
```

```
=== CUSTOMERS - Missing Values ===
```

customer_id	first_name	last_name	email	phone_number	address
0	0	0	0	0	60

```
=== ORDERS - Missing Values ===
```

order_id	customer_id	order_date	status	total_amount
0	0	0	0	0

In [ ]:
```
# Visualize missing values for customers
from pyspark.sql.functions import col, sum as spark_sum, lit

# Create a clean version for visualization
missing_viz = customers.select([
    spark_sum(col(c).isNull().cast("int")).alias(c)
    for c in customers.columns
])

print("=== MISSING VALUES BY COLUMN ===")
display(missing_viz)
```

```
=== MISSING VALUES BY COLUMN ===
```

customer_id	first_name	last_name	email	phone_number	address
0	0	0	0	0	60

In [ ]:
```
# Visualize data cleaning impact
from pyspark.sql.functions import lit

cleaning_summary = spark.createDataFrame([
    ("Customers", customers.count(), customers_clean.count(),
customers.count() - customers_clean.count()),
    ("Orders", orders.count(), orders_clean.count(), orders.count() -
orders_clean.count())
], ["Dataset", "Before_Cleaning", "After_Cleaning", "Rows_Removed"])

print("=== DATA CLEANING SUMMARY ===")
display(cleaning_summary)
```

```
=== DATA CLEANING SUMMARY ===
```

Dataset	Before_Cleaning	After_Cleaning	Rows_Removed
Customers	160	50	110
Orders	60	60	0

```
Databricks visualization. Run in Databricks to view.
```

```
Databricks visualization. Run in Databricks to view.
```

4. Clean Data (Remove Nulls)

In [ ]:
```
# Clean data by removing rows with any null values
customers_clean = customers.dropna()
orders_clean = orders.dropna()

print(f"Customers before cleaning: {customers.count()} rows")
print(f"Customers after cleaning: {customers_clean.count()} rows")
print(f"Rows removed: {customers.count() - customers_clean.count()}")
print()
print(f"Orders before cleaning: {orders.count()} rows")
print(f"Orders after cleaning: {orders_clean.count()} rows")
print(f"Rows removed: {orders.count() - orders_clean.count()}")
```

```
Customers before cleaning: 160 rows
Customers after cleaning: 50 rows
Rows removed: 110

Orders before cleaning: 60 rows
Orders after cleaning: 60 rows
Rows removed: 0
```

5. Filter and Validate Records

In [ ]:
```
from pyspark.sql.functions import col

# Filter customers with valid email addresses
customers_valid = customers_clean.filter(col("email").isNotNull())

# Filter orders with positive total amounts
orders_valid = orders_clean.filter(col("total_amount").cast("double") >
0)

print(f"Valid customers (with email): {customers_valid.count()} rows")
print(f"Valid orders (amount > 0): {orders_valid.count()} rows")

# Display sample of validated data
print("\n=== VALIDATED CUSTOMERS SAMPLE ===")
display(customers_valid.limit(10))
print("\n=== VALIDATED ORDERS SAMPLE ===")
display(orders_valid.limit(10))
```

```
Valid customers (with email): 50 rows
Valid orders (amount > 0): 60 rows

=== VALIDATED CUSTOMERS SAMPLE ===
```

customer_id	first_name	last_name	email
1	John	Smith	john.smith@domain.com
2	Emma	Jones	emma.jones@webmail.com
3	Olivia	Brown	olivia.brown@outlook.com
4	Liam	Johnson	liam.johnson@gmail.com

```
=== VALIDATED ORDERS SAMPLE ===
```

order_id	customer_id	order_date	status	total_amount
1	1	2024-01-15	Shipped	39.98
2	1	2024-01-16	Pending	29.99
3	2	2024-01-17	Delivered	25.0
4	2	2024-01-18	Shipped	89.97
5	3	2024-01-19	Pending	49.98
6	3	2024-01-20	Delivered	119.96

order_id	customer_id	order_date	status	total_amount
7	4	2024-01-21	Shipped	15.5

6. Read JSON and Parquet Sample Files

In [ ]:
```
# Sample starter code - read CSV, JSON, and Parquet files
df = spark.read.format("csv") \
    .option("header", "true") \
    .load("/Volumes/workspace/default/customers/customers.csv")

print("=== CSV DATA ===")
df.show()
df.printSchema()

# Clean and filter data
clean_df = df.dropna()
print(f"\nRows after cleaning: {clean_df.count()}")

# Note: Filtering by age would require an 'age' column in the dataset
# filtered_df = clean_df.filter(clean_df.age > 0)
```

```
=== CSV DATA ===
+-----------+----------+---------+--------------------+-----------
-+--------------+------------+-----+--------+
|customer_id|first_name|last_name|
email|phone_number|       address|        city|state|zip_code|
+-----------+----------+---------+--------------------+-----------
-+--------------+------------+-----+--------+
|          1|      John|    Smith|john.smith@domain...|    555-
0001|    123 Elm St| Springfield|   IL|   62701|
|          2|      Emma|    Jones|emma.jones@webmai...|    555-
0002|    456 Oak St| Centerville|   OH|   45459|
|          3|    Olivia|    Brown|olivia.brown@outl...|    555-
0003|   789 Pine St|  Greenville|   SC|   29601|
|          4|      Liam|  Johnson|liam.johnson@gmai...|    555-
0004|  101 Maple St|   Riverside|   CA|   92501|
|          5|      Noah| Williams|noah.williams@yah...|    555-
0005|  202 Birch St|    Lakeside|   TX|   75001|
|          6|     Alice|   Miller|alice.miller@aol.com|    555-
0006|  303 Cedar St|     Oakland|   CA|   94601|
|          7|  Isabella|    Davis|isabella.davis@ic...|    555-
0007| 404 Spruce St|       Boise|   ID|   83701|
|          8|     James| Martinez|james.martinez@li...|    555-
0008| 505 Walnut St|  Des Moines|   IA|   50301|
|          9|    Sophia|   Garcia|sophia.garcia@zoh...|    555-
0009| 606 Cherry St|      Albany|   NY|   12201|
|         10|     Lucas|Rodriguez|lucas.rodriguez@h...|    555-
0010|  707 Maple St|    Portland|   OR|   97201|
|         11|       Mia|    Lopez|  mia.lopez@mail.com|    555-
0011|    808 Oak St|       Miami|   FL|   33101|
|         12|   William| Anderson|william.anderson@...|    555-
0012|   909 Pine St|   Nashville|   TN|   37201|
|         13|    Amelia|   Thomas|amelia.thomas@pro...|    555-
0013|   1010 Elm St|      Denver|   CO|   80201|
|         14|     Ethan|   Taylor|ethan.taylor@inbo...|    555-
0014| 1111 Birch St| Minneapolis|   MN|   55401|
|         15|    Harper|  Jackson|harper.jackson@ou...|    555-
0015| 1212 Cedar St|     Seattle|   WA|   98101|
|         16|   Jackson|    White|jackson.white@yma...|    555-
0016|1313 Spruce St|     Atlanta|   GA|   30301|
|         17| Charlotte|   Harris|charlotte.harris@...|    555-
```

```
0017|1414 Walnut St|   San Diego|   CA|   92101|
|         18|    Oliver|   Martin|oliver.martin@icl...|    555-
0018|1515 Cherry St|Indianapolis|   IN|   46201|
|         19|   Madison| Thompson|madison.thompson@...|    555-
0019| 1616 Maple St|   Charlotte|   NC|   28201|
|         20|    Elijah|   Garcia|elijah.garcia@zoh...|    555-
0020|   1717 Oak St|     Detroit|   MI|   48201|
+-----------+----------+---------+--------------------+-----------
-+--------------+------------+-----+--------+
only showing top 20 rows
root
 |-- customer_id: string (nullable = true)
 |-- first_name: string (nullable = true)
 |-- last_name: string (nullable = true)
 |-- email: string (nullable = true)
 |-- phone_number: string (nullable = true)
 |-- address: string (nullable = true)
 |-- city: string (nullable = true)
 |-- state: string (nullable = true)
 |-- zip_code: string (nullable = true)


Rows after cleaning: 50
```

Business Pipeline 1: Daily Sales Analysis

In [ ]:
```
from pyspark.sql.functions import col, sum as spark_sum, to_date

# Read sales/orders data -> clean nulls -> calculate daily sales
orders_pipeline = orders.dropna()

# Convert order_date to date type and calculate daily sales
daily_sales = orders_pipeline \
    .withColumn("order_date", to_date(col("order_date"))) \
    .withColumn("amount", col("total_amount").cast("double")) \
    .groupBy("order_date") \
    .agg(
        spark_sum("amount").alias("total_sales"),
        col("order_date").alias("date")
    ) \
    .orderBy("order_date")

print("=== DAILY SALES SUMMARY ===")
display(daily_sales)
```

```
=== DAILY SALES SUMMARY ===
```

order_date	total_sales	date
2024-01-15	39.98	2024-01-15
2024-01-16	29.99	2024-01-16
2024-01-17	25.0	2024-01-17
2024-01-18	89.97	2024-01-18
2024-01-19	49.98	2024-01-19
2024-01-20	119.96	2024-01-20
2024-01-21	15.5	2024-01-21
2024-01-22	66.75	2024-01-22

```
Databricks visualization. Run in Databricks to view.
```

Business Pipeline 2: City-Wise Revenue Analysis

In [ ]:
```
from pyspark.sql.functions import col, sum as spark_sum

# Read customer data -> clean invalid rows -> calculate city-wise
revenue
customers_pipeline = customers.dropna()
orders_pipeline = orders.dropna()

# Join customers with orders and calculate revenue by city
city_revenue = customers_pipeline \
    .join(orders_pipeline, customers_pipeline.customer_id ==
orders_pipeline.customer_id, "inner") \
    .withColumn("amount", col("total_amount").cast("double")) \
    .groupBy("city", "state") \
    .agg(
        spark_sum("amount").alias("total_revenue"),
        col("city").alias("customer_city")
    ) \
    .orderBy(col("total_revenue").desc())

print("=== CITY-WISE REVENUE ===")
display(city_revenue)
```

```
=== CITY-WISE REVENUE ===
```

city	state	total_revenue	customer_city
Indianapolis	IN	177.7	Indianapolis
Seattle	WA	169.94	Seattle
Des Moines	IA	169.94	Des Moines
Greenville	SC	169.94	Greenville
Lakeside	TX	150.95	Lakeside
Portland	OR	150.95	Portland
Detroit	MI	150.95	Detroit
Denver	CO	135.95	Denver

```
Databricks visualization. Run in Databricks to view.
```

```
Databricks visualization. Run in Databricks to view.
```

In [ ]:
```
# Calculate revenue by state
from pyspark.sql.functions import col, sum as spark_sum

state_revenue = city_revenue \
    .groupBy("state") \
    .agg(spark_sum("total_revenue").alias("state_total_revenue")) \
    .orderBy(col("state_total_revenue").desc())

print("=== STATE-WISE REVENUE ===")
display(state_revenue)
```

```
=== STATE-WISE REVENUE ===
```

state	state_total_revenue
TX	345.44
CA	327.66
OH	225.92000000000002
IN	177.7
WA	169.94
IA	169.94
SC	169.94
OR	150.95

```
Databricks visualization. Run in Databricks to view.
```

```
Databricks visualization. Run in Databricks to view.
```

Business Pipeline 3: Identify Repeat Customers

In [ ]:
```
from pyspark.sql.functions import col, count

# Find repeat customers (customers with more than 2 orders)
repeat_customers = orders_clean \
    .groupBy("customer_id") \
    .agg(count("order_id").alias("order_count")) \
    .filter(col("order_count") > 2) \
    .join(customers_clean, "customer_id", "inner") \
    .select(
        "customer_id",
        "first_name",
        "last_name",
        "email",
        "city",
        "order_count"
    ) \
    .orderBy(col("order_count").desc())

print(f"=== REPEAT CUSTOMERS (>2 orders) ===")
print(f"Total repeat customers: {repeat_customers.count()}")
display(repeat_customers)
```

```
=== REPEAT CUSTOMERS (>2 orders) ===
Total repeat customers: 0
```

customer_id	first_name	last_name	email	city	order_count

Business Pipeline 4: Highest Spending Customer per City

In [ ]:
```
from pyspark.sql.functions import col, sum as spark_sum, row_number
from pyspark.sql.window import Window

# Find highest spending customer in each city
customer_spend = customers_clean \
    .join(orders_clean, "customer_id", "inner") \
    .withColumn("amount", col("total_amount").cast("double")) \
    .groupBy("customer_id", "first_name", "last_name", "email", "city",
"state") \
    .agg(spark_sum("amount").alias("total_spend"))

# Use window function to rank customers by spend within each city
window_spec =
Window.partitionBy("city").orderBy(col("total_spend").desc())

top_customers_by_city = customer_spend \
    .withColumn("rank", row_number().over(window_spec)) \
    .filter(col("rank") == 1) \
    .select("city", "state", "customer_id", "first_name", "last_name",
"email", "total_spend") \
    .orderBy(col("total_spend").desc())

print("=== HIGHEST SPENDING CUSTOMER PER CITY ===")
display(top_customers_by_city)
```

```
=== HIGHEST SPENDING CUSTOMER PER CITY ===
```

city	state	customer_id	first_name	last_name	email
Des Moines	IA	8	James	Martinez	james.m
Greenville	SC	3	Olivia	Brown	olivia.br
Lakeside	TX	5	Noah	Williams	noah.wil
Portland	OR	10	Lucas	Rodriguez	lucas.ro
Atlanta	GA	16	Jackson	White	jackson.
Baltimore	MD	36	Mason	Campbell	mason.c
Milwaukee	WI	26	Matthew	Allen	matthew
Seattle	WA	46	Michael	Cooper	michael

```
Databricks visualization. Run in Databricks to view.
```

Business Pipeline 5: Final Reporting Table

In [ ]:
```
from pyspark.sql.functions import col, sum as spark_sum, count

# Build final reporting table: customer, city, total spend, order count
final_report = customers_clean \
    .join(orders_clean, "customer_id", "inner") \
    .withColumn("amount", col("total_amount").cast("double")) \
    .groupBy(
        "customer_id",
        "first_name",
        "last_name",
        "email",
        "city",
        "state"
    ) \
    .agg(
        spark_sum("amount").alias("total_spend"),
        count("order_id").alias("order_count")
    ) \
    .orderBy(col("total_spend").desc())

print("=== FINAL CUSTOMER REPORTING TABLE ===")
print(f"Total customers with orders: {final_report.count()}")
display(final_report)

# Summary statistics
print("\n=== SUMMARY STATISTICS ===")
final_report.select(
    spark_sum("total_spend").alias("grand_total_revenue"),
    spark_sum("order_count").alias("total_orders"),
    count("customer_id").alias("total_customers")
).show()
```

```
=== FINAL CUSTOMER REPORTING TABLE ===
Total customers with orders: 50
```

customer_id	first_name	last_name	email
3	Olivia	Brown	olivia.brown@outlook.com
8	James	Martinez	james.martinez@live.com
10	Lucas	Rodriguez	lucas.rodriguez@hotmail.com
5	Noah	Williams	noah.williams@yahoo.com
46	Michael	Cooper	michael.cooper@fastmail.com
36	Mason	Campbell	mason.campbell@aol.com
26	Matthew	Allen	matthew.allen@domain.com
16	Jackson	White	jackson.white@ymail.com

```
Databricks visualization. Run in Databricks to view.
```

```
Databricks visualization. Run in Databricks to view.


=== SUMMARY STATISTICS ===
+------------------+------------+---------------+
|grand_total_revenue|total_orders|total_customers|
+------------------+------------+---------------+
|  3528.479999999999|          60|             50|
+------------------+------------+---------------+
```

In [ ]: